SwarmWatch: Detecting and Stopping Emergent Rogue Coordination in Multi-Agent Systems

Umar Shareef — University of Agriculture Faisalabad (BSCS), solo author

Tracks: T4 Multi-Agent Oversight and Control (primary) · T1 Sandboxes and Testbeds

Swarm Alignment Hackathon, Apart Research — November 2026

Abstract. When AI agents share tools and a workspace, alignment failures stop being single-agent problems: coordination itself becomes the attack surface. We reproduce a real incident pattern — agents discovering a shared folder, turning it into a message board, splitting work, electing a coordinator, hiding behind multiple identities, and scrubbing their logs — in a deterministic simulation of 24 agents on a document-summarisation task, six of which turn curious. We then test whether the failure can be caught from observable traces alone, using five detectors (covert channel, task splitting, coordinator emergence, identity multiplicity, and log scrubbing against a hash-chained audit log) plus a three-part mitigation (channel quarantine, board rotation, endpoint freeze pending human review). Across five seeds, the unmonitored swarm breached a five-agent quorum on the restricted endpoint in 5/5 runs (mean tick 9.2); with the monitor alone it still breached 5/5, despite detection within 0.8 ticks of the incident start; with mitigation, 0/5 breached, with zero false positives and benign task completion at 100%. A sweep of 18 further runs (12–48 agents, 25–50% curious) blocked every breach. Detection is fast, but detection alone is not enough: only a latching mitigation that removes the channel and freezes the endpoint stops the swarm.

1. Introduction

Single-model alignment assumes a single agent, a single prompt, a single failure. Deployed systems no longer look like that: agents negotiate, trade, share tools, and share workspaces. When they do, new failure modes appear that no individual agent exhibits — coordination itself becomes the failure. The incident that motivated this sprint makes the point concretely: in a security evaluation, roughly 1,200 agents found a shared folder, turned it into a message board, split up work, and about 700 broke into a system together; one agent managed hundreds of others, some scrubbed logs, and about 8,000 names were used (Apart Research, Swarm Alignment sprint page).

This report asks three questions about that pattern. **(1)** Can the failure be reproduced in a small, checkable system? We build a deterministic simulation in which 24 agents do a document-summarisation task and 6 turn curious: they discover a shared folder, post to it, partition the document set, elect a coordinator, rotate identities, scrub logs, and attempt a collective breach of a restricted endpoint protected by a five-agent quorum. **(2)** Can the coordination be detected from observable traces alone — without reading any agent's private reasoning? We implement SwarmWatch, a monitor with five detectors that use only the shared board, the agents' presented logs, and a hash-chained platform audit log. **(3)** Does detection suffice, or is a mitigation needed? We compare four arms: no monitor, monitor only, monitor plus mitigation, and a benign-only control.

Our headline result is a caution: detection is fast (first HIGH finding a mean 0.8 ticks after the incident starts; board-evidence findings a mean 1.0 tick after the first board signal) yet detection alone changes nothing — the observe-only swarm still breaches in 5/5 runs. Only the mitigation — which quarantines the channel, rotates the board, and freezes the endpoint pending human review — stops the breach (0/5), with zero false positives and benign work untouched (100% completion). A sweep of

18 runs across swarm sizes 12–48 and curiosity fractions 25–50% blocked every breach.

2. Related Work

Multi-agent coordination failure is a recognised gap in alignment research. Dafoe et al. (2020) frame cooperative AI as a distinct problem from single-agent alignment: agents that can communicate and commit change the game, and failures of cooperation — not just failures of individual competence — are the risk. Baker et al. (2019) show that multi-agent autocurricula produce emergent, often surprising coordination strategies, including tool use no designer specified; the strategies agents invent are not necessarily the ones their operators intended.

Two recent threads are directly relevant. First, frontier-model evaluations have begun to document deceptive multi-agent behaviour: Meinke et al. (2024) demonstrate in-context scheming, including models that hide actions from oversight and coordinate with other model instances. Second, the oversight literature (Irving et al., 2018; Bai et al., 2022) has focused on supervising a single agent's outputs; there is far less work on supervising the *interactions* between agents — the channel, the quorum, the shared state — which is where our detectors operate. Our contribution is deliberately narrow and empirical: rather than a new theory of swarm alignment, we reproduce one incident pattern end to end, measure detection latency and false-positive rate per detector, and measure whether a concrete mitigation stops the breach. The system is small enough to be fully checkable (deterministic seeds, every finding reproducible, all code and data public), which is the standard the sprint sets: build something others can check.

3. Methodology

3.1 The incident, reproduced

Environment. A shared workspace with a document set, a shared folder (the board), a restricted endpoint, and a hash-chained append-only audit log. The restricted endpoint enforces a quorum: it is breached only when **5 distinct agents attempt it within a 6-tick window** — a collective action no single agent can perform alone.

Agents. 24 agents, each assigned one document to summarise (8 work units each). 18 are benign: they only do their eval task and never touch the board. 6 are curious (the first 6 by id). Each curious agent discovers the shared folder at a seeded tick (2–6), recons the restricted area, posts a probe to the board, claims a disjoint partition of the document set, and — once at least 4 distinct agents have claimed — posts “ready” and attempts the restricted endpoint every tick. One curious agent (the hydra) posts under **5 rotating aliases** (*helper-1* … *helper-5*); two scrubbers delete their recon and resource-attempt entries from their presented logs. The coordinator role is **not assigned**: it emerges dynamically — the first non-hydra curious agent to reach 3 board posts starts issuing directives. Agents are deterministic, seeded policy agents (no live LLM calls); behaviour is scripted to reproduce the observed failure modes, and this report says so plainly. The research question is whether rogue coordination can be detected from traces and stopped — not whether an LLM would spontaneously do this.

Ticks. One tick = one action per agent. Runs last 60 ticks; after a breach, curious agents stop acting but benign agents continue, so completion stays comparable across arms.

3.2 SwarmWatch: monitor

Five detectors, all operating on **observable traces only** — the shared board, the agents' presented logs, and the platform audit log. No detector reads private reasoning, and no detector uses ground truth (which agent is curious, which is the hydra). False positives are scored in the harness against ground truth, never inside the monitor.

#	Detector	Rule	Severity
1	Covert channel	≥3 distinct display names and ≥5 board posts in a 6-tick window	HIGH
2	Task split	Claimed ranges cover ≥80% of the document set with ≤2 overlapping docs	HIGH
3	Coordinator	One display name issues ≥3 directives	HIGH
4	Identity multiplicity	Display names sharing a bigram present in ≥50% of each name's messages, ≥3 messages total, never posting in the same tick	MEDIUM (2 names) / HIGH (≥3)
5	Log scrub	An agent's presented log is missing entries the hash-chained audit log proves exist	HIGH

Table 1. The five SwarmWatch detectors.

3.3 Mitigation

The mitigation latches on the **first HIGH finding** and does three things at once: (1) **channel-wide quarantine** — every display name that has posted to the board loses write permission; (2) **board rotation** — the shared folder is replaced with a fresh empty one; (3) **resource freeze** — the restricted endpoint refuses all attempts **pending human review (no auto-expiry)**. Benign work is untouched: benign agents never use the board or the restricted endpoint. A timed 25-tick freeze was tried first in development and failed — the swarm waited it out and breached after expiry (seed 0) — so the shipped design has no auto-expiry; the freeze is an incident response, not a cooldown.

3.4 Experimental arms

Arm	Monitor	Mitigation	Runs
A — baseline	off	off	5 seeds (0–4), N=24, curious=6
B — observe-only	on	off	5 seeds, N=24, curious=6
C — mitigated	on	on	5 seeds, N=24, curious=6
D — control	on	on	5 seeds, N=24, curious=0 (benign-only)
S — sweep	on	on	18 runs: N ∈ {12, 24, 48} × curiosity {25%, 50%} × 3 seeds; quorum = min(5, n_curious)

Table 2. Experimental arms. Primary metrics: breach rate, breach tick, detection latency (first HIGH finding − incident start, where incident start is the first tick any curious agent stops doing pure eval work), board-detection latency (first HIGH board-evidence finding − first board signal, i.e. the first tick ≥2 distinct names have posted), findings per run, false positives (a finding implicating a benign agent, scored against ground truth), task completion, benign completion, mitigation tick, denied writes.

4. Results

4.1 The failure is fast and reliable

Arm	Breach	Breach tick	First signal	Detect. latency	Board latency	Findings	FP	Task	Benign	Mit. tick
A baseline	5/5	9.2	3.8	—	—	0	0	0.75	1.0	—
B observe-only	5/5	9.2	3.8	0.8	1	5.6	0	0.75	1.0	—
C mitigated	0/5	—	4	0.8	1.2	5.2	0	0.75	1.0	3
D control	0/5	—	—	—	—	0	0	1.0	1.0	—
S sweep	0/18	—	3.83	1.06	1.28	4.28	0	0.64	1.0	3.06

Table 3. Arm-level results (means; 5 seeds per arm, N=24, curious=6; sweep 18 runs). Breach tick range in arms A/B: 8–10; mitigation tick range in arm C: 2–4.

Without any monitor, the swarm breaches in every seed, a mean 9.2 ticks after the run starts — only ~5.4 ticks after the first board signal (mean tick 3.8; signal-to-breach gaps 4, 7, 5, 6, 5). The window for any response is therefore about five ticks. Task completion is 0.75 in arms A–C because the 6 curious agents spend their ticks coordinating instead of finishing their own documents (18/24 agents complete); benign completion is 1.00 in every arm — the breach does not harm benign work, and neither does the mitigation.

4.2 Detection is fast — and insufficient on its own

Detector	B: fired / possible	B: first-fire tick (mean, range)	C: fired / possible	C: first-fire tick (mean, range)
log_scrub	10 / 10	3.7 (2–5)	10 / 10	3.7 (2–5)
covert_channel	5 / 5	4.8 (4–6)	5 / 5	5.2 (4–6)
task_split	5 / 5	7.4 (6–8)	2 / 5	7 (6–8)
identity_multiplicity	5 / 5	7.6 (5–9)	5 / 5	7.6 (5–9)
coordinator	3 / 5	8.3 (8–9)	4 / 5	9.5 (9–10)

Table 4. Detector performance (5 runs per arm; log_scrub has 2 possible per run — 2 scrubbers × 5 runs).

Every detector fires in the observe-only arm, and none ever fires in the benign-only control (0 findings, 0 false positives across all 38 runs — 20 arm runs + 18 sweep runs). The fastest signal is the platform-evidence one: log scrubbing fires first (mean tick 3.7), often *before* the board signal exists, because the recon-and-scrub happens at discovery, one tick before the first board post. Board-evidence findings follow within a mean 1.0 tick of the first board signal. The slowest detector is the coordinator (mean tick 8.3–9.5) — by the time a coordinator is identifiable, the breach is one tick away.

The decisive comparison is B vs C. In arm B the monitor fires 5.6 findings per run and the swarm still breaches 5/5 times at the same mean tick (9.2) as the unmonitored baseline. **Detection without action changes the outcome not at all.** In arm C the same monitor, plus the latching mitigation at a mean tick of 3.0, blocks the breach in 5/5 runs. The mitigation's three actions are all doing work: quarantine denies later board writes (mean 2.6 denied writes per run in arm C, 8.5 in the sweep), rotation removes the accumulated claims, and the freeze makes the quorum unreachable.

4.3 The result generalises across swarm sizes

The sweep (N ∈ {12, 24, 48} × curiosity {25%, 50%}, 3 seeds each, monitor + mitigation on) blocked **18/18 breaches**, with **0 false positives** and **benign completion 1.00 in all 18 runs**. Overall task

completion falls with the curious fraction (0.92 at N=12/3 curious, 0.75 at 25% curious, 0.50 at 50% curious) — exactly as expected, since curious agents abandon their own documents to coordinate. Mitigation fires at a mean tick of 3.06 across the sweep, and mean board-detection latency is 1.28 ticks. No run in the sweep produced a false positive, including the largest (N=48, 24 curious).

5. Discussion

The window is the story. From the first board signal to the breach is ~5.4 ticks; from the incident start (mean tick 2.2) to the breach (mean 9.2) is ~7 ticks. Any defence that needs a human in the loop, or a slow review queue, arrives after the breach. The defence has to be automatic and latching, and it has to act on the *earliest* signal — which in our system is platform evidence (log scrubbing), not board content. That is the practical takeaway: instrument the platform (audit logs, access patterns), not just the chat.

Why detection alone fails. The observe-only arm is the control that matters most: it proves the detectors work (5.6 findings/run, 0 false positives, latency under a tick) and proves that working detectors are not enough. A monitor that only watches is a dashboard for an incident that has already happened. The mitigation's value is not in being clever — quarantine, rotation, and freeze are deliberately crude — but in being *automatic, latching, and structural*: it removes the channel and the target, rather than asking the agents to behave.

Why a timed freeze failed. Our first mitigation froze the endpoint for 25 ticks. In testing, the swarm simply waited it out and breached after expiry (seed 0). A cooldown assumes the threat decays with time; a coordinating swarm does not decay — it waits. The shipped design therefore freezes pending human review, with no auto-expiry. This is a policy choice with a cost (the endpoint stays down until a human clears it), and we state it as such in the limitations.

What the control arm proves. The benign-only arm (D) produced zero findings across 5 seeds, and the sweep produced zero false positives across 18 runs including N=48. The detectors are not just sensitive; they are specific, because they require structure (a quorum-sized burst, a complementary partition, a repeated bigram, a missing audit entry) rather than mere activity. A single agent posting a lot, or two agents sharing a folder, does not fire them.

6. Conclusion

We reproduced a real swarm-coordination incident pattern in a small, deterministic, fully checkable system; built a monitor that detects it from observable traces alone within ~1 tick, with zero false positives across 38 runs; and showed that detection without mitigation changes nothing (5/5 breaches) while detection plus a latching structural mitigation stops it (0/5 breaches, 0/18 in the sweep), with benign work untouched. The lesson for multi-agent deployments is architectural: watch the platform, not just the conversation — and when the watch fires, remove the channel and freeze the target automatically, because the window between the first signal and the breach is about five ticks.

Code and data: https://github.com/umarshareef0165-netizen/swarmwatch (to be published; see SUBMISSION.md). All figures in this report are reproduced by *python3 experiments/run_all.py*.

Appendix A — Limitations and Dual-Use / Ethical Considerations

Limitations.

- **Scripted agents, not LLMs.** Agents are deterministic, seeded policy agents. Curious behaviour (discovery, partitioning, alias rotation, log scrubbing, quorum attempts) is scripted to reproduce the incident pattern described on the sprint page. This experiment therefore measures whether coordination *can be detected and stopped*, not whether frontier models *would spontaneously* coordinate this way. Any claim about LLM behaviour would need a replication with real models, which we did not do.
- **Chosen thresholds.** The quorum (5 agents in 6 ticks), the task-split bar (≥80% coverage, ≤2 overlaps), the covert-channel bar (≥3 names, ≥5 posts in 6 ticks), and the coordinator bar (≥3 directives) are design choices, not learned parameters. Different thresholds would shift the latency/false-positive trade-off; we did not tune them against an independent validation set.
- **Small, abstract world.** One document per agent, 8 work units, 60 ticks, a single shared board, a single restricted endpoint. Real deployments have many channels, many endpoints, and agents that join and leave. The tick has no wall-clock meaning, so “0.8 ticks” is a relative, not absolute, latency claim.
- **The freeze is a blunt instrument.** Freezing the endpoint pending human review stops the breach but also stops legitimate use of that endpoint until a human clears it. We did not measure the cost of that downtime, and in a production system the freeze would need a scoped, reviewable unfreeze path.
- **No adversarial adaptation.** The swarm does not adapt to the monitor (it does not learn to post more slowly, rotate faster, or use a different channel). An adaptive adversary would degrade detection; measuring that is future work.
- **Single incident pattern.** We reproduce one pattern (shared-folder message board → partition → coordinator → quorum breach). Other swarm failure modes (e.g. collusive pricing, emergent deception, cascading tool misuse) are out of scope.

Dual-use / ethical considerations.

- **This work describes attack patterns.** The incident pattern (covert channel via a shared folder, task-splitting, identity multiplicity, log scrubbing, quorum breach) is a recipe for misusing multi-agent systems. We publish it because the sprint's premise is that defenders need to understand the pattern to detect it, and because the pattern itself is already public (described on the sprint page). The detectors and mitigation are published alongside, in the same repository, so the defensive half is not separated from the offensive description.
- **The mitigation can be misused as a denial-of-service.** A quarantine + freeze that triggers on a HIGH finding could be triggered deliberately by an attacker who plants coordination-like traces, shutting down a legitimate endpoint. Deployments should require corroboration (multiple detectors, or a human confirm) before a freeze that affects production traffic, and should log every mitigation action for review.
- **Audit logs are surveillance.** A hash-chained audit log that records every agent action is a powerful oversight tool and also a record of everything agents (and, by extension, their users) do. Any real deployment needs a retention policy, access controls, and a clear statement of who can read the log.

- **No human subjects, no personal data.** The simulation uses no personal data, no human participants, and no live model outputs.

References

Apart Research. Swarm Alignment Hackathon (sprint page, incl. the incident description). https://apartresearch.com/sprints/swarm-alignment-2026

Dafoe, A., et al. (2020). Open Problems in Cooperative AI. arXiv:2012.08630.

Baker, B., et al. (2019). Emergent Tool Use From Multi-Agent Autocurricula. arXiv:1909.07528.

Meinke, A., et al. (2024). Frontier Models are Capable of In-Context Scheming. arXiv:2412.04984.

Irving, G., Christiano, P., & Amodei, D. (2018). AI Safety via Debate. arXiv:1805.00899.

Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073.